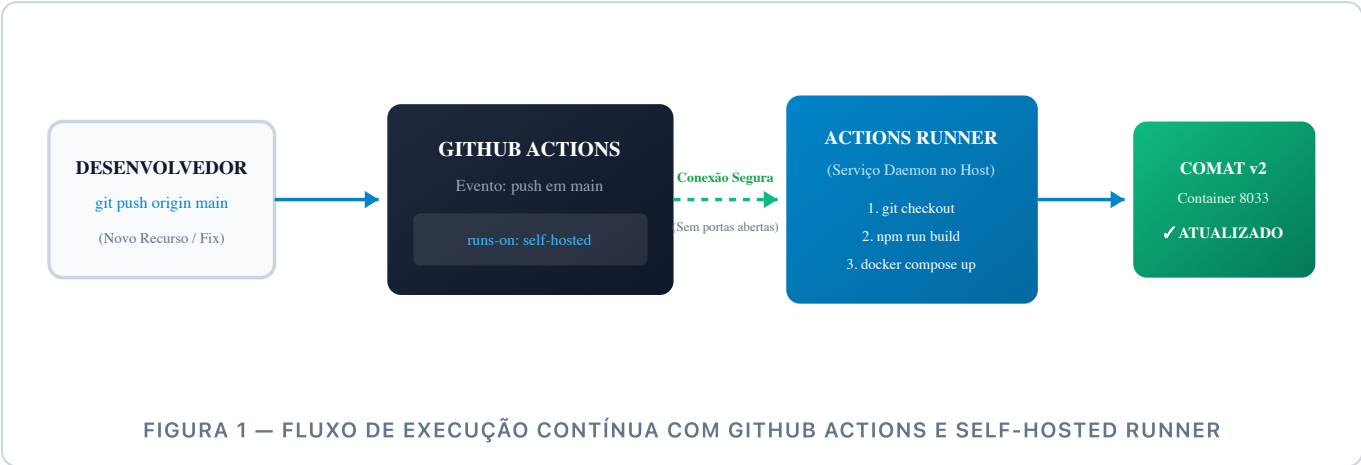


Automação de Deploy Contínuo com GitHub Actions (Self-Hosted Runner)

Este guia documenta o ciclo automatizado de atualização contínua do sistema **COMAT v2**. Ao enviar uma alteração de código para o repositório GitHub (`git push origin main`), o servidor Linux no ambiente local ou corporativo detecta o evento e atualiza os containers Docker em tempo real sem intervenção manual.

1. Topologia do Pipeline de Deploy



2. Por Que Usar Self-Hosted Runner?

SEM PORTAS ABERTAS 100% Blindado Comunicação outbound. Funciona em rede interna e residencial.	ZERO CRON POLLING Reativo / Push Dispara no milissegundo do push sem gastar CPU com loops.	LOGS NO GITHUB Aba Actions Acompanhe o build e o status diretamente no navegador.
--	--	---

3. Etapa 1: Obter o Token de Registro no GitHub

1. Acesse: <https://github.com/ubsramos/COMAT-v2/settings/actions/runners/new>
2. Selecione **Linux** e arquitetura **x64**. O GitHub fornecerá os comandos com o token temporário.

Instalação do Runner e Configuração do Serviço no Linux

1. Comandos de Instalação no Terminal do Servidor

Conecte via SSH no servidor Linux (ssh uli@192.168.15.4) e execute os passos abaixo:

```
# 1. Criar pasta do runner na home do usuario
mkdir -p ~/actions-runner && cd ~/actions-runner

# 2. Baixar o pacote oficial do GitHub Runner
curl -o actions-runner-linux-x64-2.322.0.tar.gz -L https://github.com/actions/runner/releases/download/v2.322.0/actions-runner-linux-x64-2.322.0.tar.gz

# 3. Descompactar o instalador
tar xzf ./actions-runner-linux-x64-2.322.0.tar.gz

# 4. Registrar o runner no repositório (substitua pelo token do GitHub)
./config.sh --url https://github.com/ubsramos/COMAT-v2 --token SEU_TOKEN_AQUI
```

2. Configurar o Runner como Serviço Permanente (Systemd Daemon)

Para garantir que o Runner execute em segundo plano e inicie sozinho no boot do servidor:

```
cd ~/actions-runner
sudo ./svc.sh install
sudo ./svc.sh start
```

3. Estrutura do Workflow do GitHub Actions (.github/workflows/deploy.yml)

O arquivo de automação já integrado no projeto executa as seguintes etapas no servidor:

ETAPA (STEP)	COMANDO EXECUTADO NO SERVIDOR	OBJETIVO
1. Checkout	actions/checkout@v4	Baixa a versão mais recente do código
2. Build Frontend	cd frontend && npm install && npm run build	Compila o React 19 em bundle SPA/PWA
3. Atualizar App	cp -r frontend/dist/* DOCKER-DISTRIB/app/public/	Copia assets para o volume Docker
4. Docker Reload	cd DOCKER-DISTRIB && docker compose up -d --build	Aplica as alterações com zero downtime

4. Comandos de Manutenção e Diagnóstico do Runner

```
# Verificar status do serviço:    sudo ./svc.sh status
# Reiniciar o runner:            sudo ./svc.sh restart
# Parar o runner:                sudo ./svc.sh stop
# Desinstalar serviço:           sudo ./svc.sh uninstall
```

Como Disparar o Deploy: No seu Mac, basta commitar suas alterações e dar `git push origin main`. A aba **Actions** no GitHub mostrará o deploy sendo executado em tempo real!